means any cache can store the request. Also we commonly want to set `max-age` which is how long the cache will store the object before invalidating the cache. The `max-age` value is set to seconds with a maximum possible value of `31536000` which is one year. You can do this in your rails application by setting
`config.static_cache_control = "public, max-age=31536000"`
Now when your application serves an asset in production, the CDN will store the asset for up to a year. Since most CDNs also cache headers of the request, this `Cache-Control` will be passed along to all future browsers seeking this asset, the browser then knows that it can store this asset for a very long time before needing to re-request it.

4.4.2.4 CDNs and URL based Cache Invalidation

Most CDNs will cache contents of an asset based on the complete URL. This means that a request to

```
http://mycdnsubdomain.fictional-cdn.com/assets/smile-123.png
```

Will be a completely different cache from

```
http://mycdnsubdomain.fictional-cdn.com/assets/smile.png
```

If you want to set far future `max-age` in your `Cache-Control` (and you do), then make sure when you change your assets that your cache is invalidated. For example when changing the smiley face in an image from yellow to blue, you want all visitors of your site to get the new blue face. When using a CDN with the Rails asset pipeline `config.assets.digest` is set to true by default so that each asset will have a different file name when it is changed. This way you don't have to ever manually invalidate any items in your cache. By using a different unique asset name instead, your users get the latest asset.

5 Customizing the Pipeline

5.1 CSS Compression

One of the options for compressing CSS is YUI. The YUI CSS compressor provides minification.
The following line enables YUI compression, and requires the `yui-compressor` gem.
`config.assets.css_compressor = :yui`
The other option for compressing CSS if you have the sass-rails gem installed is

```
config.assets.css_compressor = :sass
```

5.2 JavaScript Compression

Possible options for JavaScript compression are `:closure`, `:uglifier` and `:yui`. These require the use of the `closure-compiler`, `uglifier` or `yui-compressor` gems, respectively.
The default Gemfile includes uglifier. This gem wraps UglifyJS (written for NodeJS) in Ruby. It compresses your code by removing white space and comments, shortening local variable names, and performing other micro-optimizations such as changing `if` and `else` statements to ternary operators where possible.
The following line invokes `uglifier` for JavaScript compression.
`config.assets.js_compressor = :uglifier`
You will need an ExecJS supported runtime in order to use `uglifier`. If you are using Mac OS X or Windows you have a JavaScript runtime installed in your operating system.

The `config.assets.compress` initialization option is no longer used in Rails 4 to enable either CSS or JavaScript compression. Setting it will have no effect on the application. Instead, setting `config.assets.css_compressor` and `config.assets.js_compressor`will control compression of CSS and JavaScript assets.